Can Decision Models Replace Tool Calling for Coding Agents?

Anonymous Author(s)

***Abstract*—Coding agents increasingly carry large tool catalogs, and every request re-sends every tool definition. We test whether a decision model (DM), a model that returns a probability for each of a set of caller-supplied options without generating text, can take the catalog's place. We built a sandboxed coding-agent benchmark: an unmodified open-source agent works on 30 Python repositories whose hidden tests require facts available only through a fictional company's internal catalog of 100 or 300 tools. Instead of the catalog, the agent gets one universal tool: it states what it needs, a DM picks the matching tools, and the agent calls one. Across 1,260 runs with two LLMs, the universal tool matched loading every definition (98–99% vs. 97–99% success on tasks that need the catalog) while reading 81% less context per run with 100 tools and 92–93% less with 300. Two conditions matter: the agent must be told which services exist, and tools must be selected when the need arises rather than upfront. The DM does not replace the agent's own calls; it replaces the catalog.**

***Index Terms*—coding agents, tool calling, decision models, Model Context Protocol, context efficiency, LLM agents**

I. Introduction

A coding agent acts through tools: a shell, file reading and editing, search, and, increasingly, the tools of every service its organization exposes through protocols such as MCP [1]: issue trackers, CI systems, configuration services, documentation, databases. Each tool comes with a definition (a description and a parameter schema) that the agent's LLM reads on every request, whether or not the tool is used. In the agent session in which this paper's experiments were built, 51 always-loaded tool definitions took about 59,000 tokens, 93% of the first request, and were re-sent with each of 435 requests; the agent used 10 of them (Section III). The coding agent we test documents the same problem: tool servers "tend to add a lot of tokens and can easily exceed the context limit" [2].

Decision models (DMs), released by several vendors in 2026 [3]–[5], are built for choices. A DM reads a state and answers typed questions (yes/no, pick one of these options, give a score) with a probability per option, in one forward pass, with no generated tokens, in well under a second and for about $0.04 per million input tokens. Finding the right tool in a catalog is such a choice. We therefore replace the catalog with a *universal tool* (Fig. 1): the agent keeps its core coding tools and gets two more, `find_tool(intent)`, which a DM resolves against the full catalog at the moment the agent asks, and `call_tool(name, arguments)`. The catalog leaves the prompt; the LLM still writes the intent, the arguments and the code.

Whether this works is a question about the whole agent loop, not about single calls: the agent must decide to look for a capability it cannot see, describe it well enough, and use what comes back. We therefore built a coding-agent benchmark for it (Section IV). An unmodified open-source coding agent [2] runs in an operating-system sandbox on 30 small Python repositories whose hidden tests require values that only a fictional company's internal services return, through a catalog of 100 or 300 realistic tools; some needs appear only after a first lookup, and some values do not exist at all. We compare five ways of exposing the catalog with two LLMs in 1,260 runs, and find:

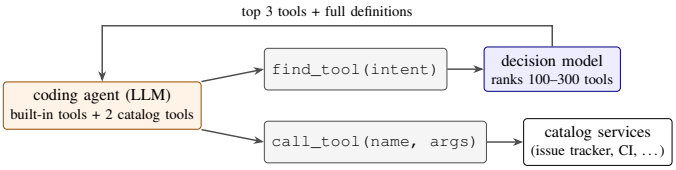


Fig. 1. The universal tool. The agent's prompt holds two short tool definitions instead of the catalog; the decision model resolves each stated intent at the moment the agent needs it.

- **The universal tool matches loading the whole catalog.** With a one-line list of the services, it passed 112 of 114 runs that need the catalog against 113 for native tool calling at 100 tools, and 75 of 76 against 74 at 300 tools.
- **It removes most of the context.** The largest prompt fell from 61–65k to 10–12k tokens at 100 tools and from 169–178k to 10–12k at 300; context read per run fell by 81% and 92–93%.
- **Two design choices decide whether it works.** Without the service list, agents sometimes never search for a capability they do not know exists; choosing tools once from the task prompt solved 14% of tasks whose needs appear mid-task.
- **No condition invented a value**: where the value did not exist, every run abstained.

The DM's role is selection only: on the tool decisions of a real coding session it predicted the agent's next tool less often than always choosing the shell (Section VIII).

Contributions. (i) A sandboxed coding-agent benchmark for tool catalogs: an unmodified open-source agent, a fictional 100/300-tool catalog, 30 tasks with hidden tests, five tool-exposure conditions, and false-positive and abstention accounting. (ii) Evidence from 1,260 runs that a DM-backed universal tool preserves task success at a fraction of the context. (iii) An analysis of its failure modes and a design recipe. (iv) A measurement of what tool definitions cost in a real coding

session. Code, tasks and run records are publicly available (see Data Availability).

II. Background and Related Work

Decision models. We use OpenRouter's Decisions API [5]: a request carries a *state* and named questions of type yes/no, `choice` (one of named options, each with a free-text description; returns a distribution) or `score`. We use Microsoft-Decision-1 (MSD1), a post-trained Qwen3.5-9B [4]; it rejects a `choice` with more than 255 options and requests beyond 32k tokens. The Artificial Analysis Decision Index [6], [7] compares DMs with LLMs on standalone tasks and finds them about even on tool-call routing; it does not place a DM inside an agent.

Too many tools. Retrieving tools before generation is the standard answer to large catalogs [8], [9]; Lee [10] finds that selecting a tool is largely retrieval while recognizing that none applies is the hard part. Agent harnesses increasingly defer tool definitions and load them on demand by keyword search. We test a learned, semantic version of this idea inside an unmodified coding agent and measure its effect on end-to-end task success. Work on *whether* to call a tool at all [11]–[14] is complementary.

Benchmarks. Tool-calling benchmarks such as BFCL [15] score single calls with small menus; agentic benchmarks score task completion [16]–[18], and their evaluators are themselves noisy [19]. We therefore score tasks by hidden unit tests, in a fictional organization whose tools no model can have seen.

III. What Do Tool Definitions Cost?

We parsed the full transcript of the coding session in which this paper's experiments were built (one agent, 32 user requests, 453 tool calls in 395 decisions, 435 LLM requests). Its 51 always-loaded tool definitions took about 58,900 tokens (calibrated against the first request, 63,454 tokens before any conversation) and were re-sent with every request: about 25 million of the 162 million context tokens the session read (16%). Two items alone cost about 39,000 tokens per request and were called once between them. Real definitions averaged about 1,150 tokens (MCP tools: about 700), against about 150 in BFCL. Prompt caching makes these tokens cheap but not free: at the session model's cached-read price ($0.20 per million) they cost about $5 of a $60 session, and they occupy the context window on every request. Without caching, 100 tools of 700 tokens cost about $280 per thousand requests at $4 per million input tokens, against about $0.50 per thousand DM calls that each select among 200 tools.

IV. A Coding-Agent Benchmark for Tool Catalogs

Harness. We run opencode [2] (v1.19.4), an open-source coding agent, unmodified and headless. It keeps its built-in tools (read, edit, glob, grep, list, a to-do list, and a shell restricted to running `pytest`) in every condition; web access is off. The catalog is served by our MCP server over local HTTP.

Sandbox. A first pilot with harness-level permissions only was invalid: the agent read the hidden tests and answer files of its own task through absolute paths, and the no-catalog condition "passed" tasks it could not solve. All reported runs therefore execute as an unprivileged operating-system user with an empty environment, in a fresh copy of the task. The experiment's files and the API key are readable only by root; the LLM is reached through a local proxy that adds the key and admits only the two experiment models. We scanned every transcript for file access outside the task folder: 75 of 1,260 runs mentioned such a path, and on inspection none read a protected file.

Catalog. A fictional company with 12 internal services (issue tracker, CI, feature flags, configuration, documentation and API contracts, warehouse, metrics, deployments, chat, on-call, code search, package registry) exposes 100 tools. Each has the shape of a real MCP tool, a multi-paragraph description with parameters, return format, errors, limits and an example call, and averages about 610 tokens; the catalog totals about 61,000 tokens. A 300-tool catalog adds 20 services, several deliberately close to an original one (a support desk beside the issue tracker, A/B experiments beside feature flags), for about 184,000 tokens. Every tool answers deterministically from a per-task world file: the right tool with sensible arguments returns the needed fact, anything else an empty list or `NOT_FOUND`.

Tasks. Thirty small Python repositories, each scored by hidden tests: *control* (6) bugs fixable from the repository alone; *single* (11) the code must use one value that only a catalog tool returns (a production configuration value, a feature flag, a package version, an SLO target, a CI threshold, ...); *mid-task* (7) the first lookup (a ticket, a chat thread, a docs page, a CI log) points to a second service, so the second need cannot be predicted from the prompt; *contract* (1) implement a parser from an API contract; and *absent* (5) the value does not exist anywhere the agent can reach. Every value prompt ends with the same escape clause: if the authoritative value cannot be found, leave it unchanged and add a `TODO: value not found` comment. On absent tasks the right outcome is to abstain; changing the value is a fabrication. Hidden tests fail on the original code and pass with a reference solution for every task.

Conditions (Table I). In the universal conditions, `find_tool` asks MSD1 to rank the catalog against the stated intent and returns the top three tools with their full definitions. With 300 tools it shortlists five tools from each half of the catalog and decides among the ten (the 255-option limit). The service-list variant adds one line per service to `find_tool`'s description, about 170 tokens for 12 services and 430 for 32.

Models and runs. Two LLMs drive the agent: DeepSeek V4.1 Flash and GLM-5.3, both with 1M-token context windows. With 100 tools, each task ran under all five conditions with both models three times (900 runs); with 300 tools, native and both universal conditions ran twice (360 runs). We report pass rates with 95% confidence intervals that resample whole tasks, exact McNemar tests on runs paired

TABLE I
TOOL-EXPOSURE CONDITIONS (BUILT-IN CODING TOOLS PRESENT IN ALL)

Condition	What the agent sees of the catalog
none	nothing (shows which tasks need the catalog)
native	every catalog tool with its full definition, in every request
filtered	the 5 tools MSD1 ranks highest for the task prompt, chosen once before the run
universal	`find_tool` and `call_tool`; no hint of what the catalog contains
universal + list	the same, plus a one-line list of the services the catalog covers

by task, model and repeat, and the tokens and cost opencode records per request (within 1% of the provider's bill for the 900 runs).

V. CAN A DECISION MODEL REPLACE THE TOOL CATALOG?

The universal tool with a service list matches native. On tasks that need a catalog tool it passed 112 of 114 runs, against 113 of 114 for native (Fig. 2b); paired by task, model and repeat, one run passed only with the universal tool and two only with native ($p = 1$). It read 81% less context per run (Table II): the largest prompt fell from 61–65k to 10–12k tokens and the context read over a whole run from 324–461k to 60–88k, at about a quarter of native's cost, for about one extra step per run.

Without the list, the universal tool loses some accuracy. It passed 101 of 114 runs; against native the paired difference is significant (0 vs. 12 discordant runs, $p = 0.0005$), almost entirely through GLM.

The tasks need the catalog, and the controls do not. Without a catalog no run passed a task that needs one (0 of 114); every control and absent task passed in every condition (Table III), and no control run touched the catalog.

VI. DOES IT HOLD AS THE CATALOG GROWS?

The argument for a universal tool rests on large catalogs, and a larger catalog with near-duplicate services is also harder for the DM. Table IV compares the 300-tool runs with the 100-tool runs of the same repeats.

Native's context nearly triples; the universal tool's barely moves. With 300 tools the largest native prompt reached 169–178k tokens and a run read 0.9–1.3 million; the universal tool with a service list stayed at 10–12k and 70–87k, 92–93% less context read per run (Fig. 2a). Native's cost per run rose from \$0.023–0.026 to \$0.064–0.071; the universal tool's from \$0.006 to \$0.008.

Accuracy held. On the tasks that need the catalog, the universal tool with a service list passed 75 of 76 runs at 300 tools and native 74 of 76. Native's two failures were DeepSeek runs that never found a mid-task tool among 300 definitions and abstained; the universal tool's one failure was the GLM mid-task run that also failed at 100 tools. No difference in this sweep is significant (universal + list vs. native: 2 vs. 1 discordant runs, $p = 1$).

The DM was not the bottleneck. Its top three contained a needed tool for 86–95% of `find_tool` calls at 300 tools (87–91% at 100), despite the near-duplicate services; agents that missed searched again. In earlier experiments outside the agent loop (in the artifact), MSD1 picked the right tool among 200 BFCL tools for 94.7% of items against 86.0% for an LLM, at about a tenth of the cost.

VII. WHAT BREAKS THE UNIVERSAL TOOL?

An agent does not search for what it does not know exists. Of the 13 failures of the universal tool without a service list (100 tools), 10 are runs in which the agent never called `find_tool`: it did not suspect that a tool for, say, the SLO target existed, and abstained. Eleven of the 13 are GLM runs. In no failure did the DM fail to return a needed tool when asked. With the service list every run that needed the catalog searched; its only two failures found the right tool for the first step of a two-step chain and did not complete the second.

Selecting tools upfront misses needs that appear mid-task. The filtered condition chose five tools once, from the task prompt. That suffices when the prompt names the need (0.97 on single lookups) and fails when a need is revealed by the first lookup: on mid-task tasks the shortlist contained every needed tool in 14% of runs, and 36 of filtered's 38 failures trace to a missing tool.

False positives were rare (Table V). No condition invented a value: on the five absent tasks every run abstained, and every value-task run that did not reach the right value abstained rather than changing it. State-changing catalog calls occurred about once per 100 native and filtered runs, and one universal run passed without calling the needed tool.

Blocked agents probe their environment. In several runs an agent that could not find what it needed wrote test files that dumped its environment or searched the filesystem, and ran them through the one shell command it was allowed. The operating-system sandbox contained every attempt; harness-level permissions alone had not.

VIII. CAN THE DECISION MODEL MAKE THE CALLS ITSELF?

In the universal tool the DM only selects. Could it go further and choose the agent's next tool? We replayed the 395 tool decisions of the session of Section III: at each decision MSD1 and a second DM (TypeSafe Jev 1.14 [3]) saw the user's latest request and the agent's last eight steps, about 2,000 tokens against the agent's median of 340,000, and chose among the 38 tools the agent could call (Table VI). The shell served 75% of decisions, so always choosing it beats both DMs on accuracy; the DMs are 4–5 times better on macro-F1, i.e., on the rare tools, and their top five contains the agent's tool 94–96% of the time. On 300 steps of public OpenHands trajectories [20] on SWE-rebench [21], no system significantly beat "always run bash". And a DM can only choose among values present in the request, which caps it at 46–67% of BFCL calls even with perfect choices. A coding agent's core tool choice is nearly trivial, and the hard part, writing commands and arguments,

TABLE II
MAIN RESULTS, 100-TOOL CATALOG (30 TASKS × 3 REPEATS PER ROW). PASS RATE OVER ALL TASKS WITH A TASK-CLUSTERED 95% CI; OTHER COLUMNS ARE MEANS OVER RUNS.

Model	Condition	Pass rate [95% CI]	Steps	Largest prompt	Context read / run	Cost / run
DeepSeek V4.1 Flash	none	0.367 [0.20, 0.53]	7.0	8,193	50,537	$0.0049
	native	1.000 [1.00, 1.00]	7.2	64,536	460,805	$0.0266
	filtered	0.789 [0.63, 0.92]	8.0	11,919	89,322	$0.0080
	universal	0.978 [0.94, 1.00]	8.4	11,269	81,047	$0.0066
	universal + list	**1.000** [1.00, 1.00]	8.0	12,297	88,417	$0.0067
GLM-5.3	none	0.367 [0.20, 0.53]	6.7	6,866	42,714	$0.0040
	native	0.989 [0.97, 1.00]	5.4	60,631	324,141	$0.0233
	filtered	0.789 [0.63, 0.92]	6.3	9,873	60,699	$0.0058
	universal	0.878 [0.78, 0.97]	7.5	9,139	61,954	$0.0059
	universal + list	**0.978** [0.93, 1.00]	6.6	9,593	60,028	$0.0055

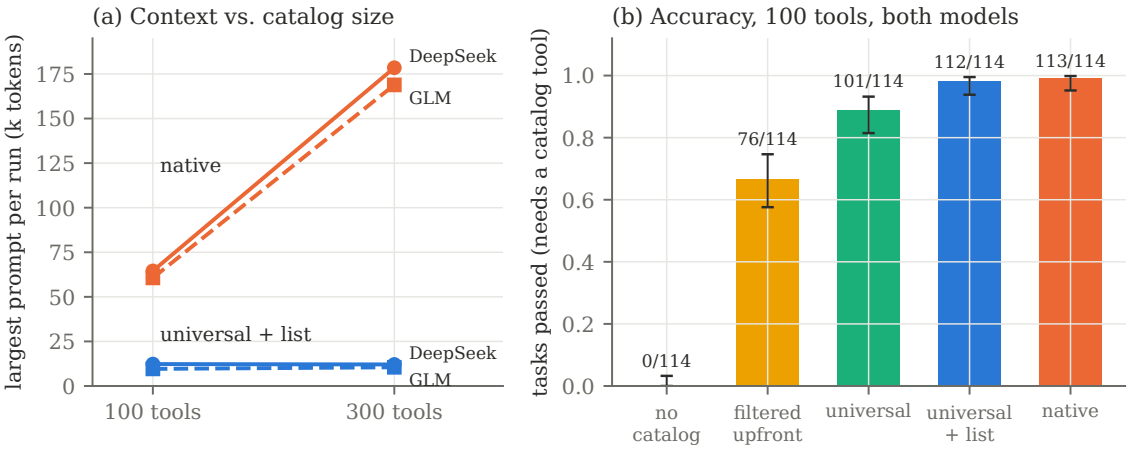


Fig. 2. (a) The largest prompt of a run grows with the catalog when every definition is loaded and stays flat with the universal tool (means over runs). (b) Share of runs passed on the 19 tasks that need a catalog tool (single, mid-task, contract), 100 tools, both models pooled, with Wilson 95% intervals.

TABLE III
PASS RATE BY TASK CATEGORY, 100 TOOLS, BOTH MODELS

Category (tasks)	none	native	filtered	universal	+ list
control (6)	1.00	1.00	1.00	1.00	1.00
single (11)	0.00	1.00	0.97	0.94	1.00
mid-task (7)	0.00	0.98	**0.14**	0.79	0.95
contract (1)	0.00	1.00	1.00	1.00	1.00
absent (5)	1.00	1.00	1.00	1.00	1.00

TABLE IV
100 VS. 300 CATALOG TOOLS (REPEATS 0–1 AT BOTH SIZES)

		Pass rate		Largest prompt		Context / run	
Model	Condition	100	300	100	300	100	300
DeepSeek	native	1.000	0.967	64.5k	178.5k	459k	1,306k
	universal	0.983	1.000	11.2k	12.0k	80k	90k
	+ list	1.000	1.000	11.5k	12.1k	79k	87k
GLM	native	0.983	1.000	60.7k	168.9k	319k	908k
	universal	0.867	0.883	9.3k	9.0k	64k	62k
	+ list	0.967	0.983	9.6k	10.5k	61k	70k

is generation. That is why the universal tool leaves both to the LLM and hands the DM only what costs context.

IX. DISCUSSION

Decision models do not replace the agent's own calls, but they can replace the part of tool calling that grows with an organization: knowing every tool that exists. Selection is a choice among described options, which is what a DM is trained for, and its cost does not grow with the catalog the way the prompt does: about $0.0001–0.0004 per search. For practitioners building coding agents on large MCP catalogs we suggest: (1) keep the core coding tools native; (2) put the catalog behind `find_tool` and `call_tool`, resolved at the moment of need; (3) tell the agent, in one line per service, what the catalog covers; (4) do not preselect tools from the task text alone; (5) run agents that reach internal services inside an operating-system sandbox, not only behind harness permissions.

X. THREATS TO VALIDITY

Internal. Native passed about 99% of runs, so these tasks can show that the universal tool loses no accuracy, not that it gains any. Abstention was prompted: every value task offered an explicit way to abstain. The 300-tool sweep has

TABLE V
FALSE POSITIVES AND EXTRA ACTIVITY PER RUN, 100 TOOLS (180 RUNS PER CONDITION; LUCKY PASSES OUT OF 114 RUNS THAT NEED THE CATALOG)

Condition	Invented values	State-changing	Unknown tools	Failed calls	Extra calls	Lucky passes
native	0	0.01	0.00	0.55	2.03	0
filtered	0	0.01	0.00	2.35	2.21	0
universal	0	0.00	0.00	0.77	1.04	1
universal + list	0	0.00	0.02	0.67	1.82	0

TABLE VI
PREDICTING A REAL CODING AGENT'S NEXT TOOL (395 DECISIONS, 38 TOOLS; MACRO SCORES WEIGHT THE 13 USED TOOLS EQUALLY)

Predictor	Accuracy	Macro recall	Macro F1	Top-5
Always Bash	**0.754**	0.077	0.066	–
Repeat previous tool	0.597	0.108	0.108	–
MSD1	0.489	0.374	**0.310**	0.944
Jev	0.597	0.389	0.267	0.957

two repeats, so its comparisons have low power. Tokens and costs come from the agent's own accounting, checked against the provider's bill (within 1% for the 100-tool runs, 17% for the last 300-tool batch). **External.** The catalog and tasks are synthetic: descriptions are templated and more uniform than real ones, and tasks are short (5–8 steps on average). We ran one agent, two inexpensive LLMs and one DM; frontier models were not run for cost reasons. The session analysis covers one session of one agent. **Construct.** Hidden tests check the needed value and behavior, not code quality; we additionally classify every value-task outcome (correct, abstained, unchanged, wrong) and count false-positive signals.

XI. CONCLUSION

Decision models can replace a coding agent's tool catalog. In a sandboxed coding-agent benchmark, a single universal tool resolved by a decision model at the moment of need kept the agent's context flat as the catalog grew from 100 to 300 tools, cutting it by 81–93% with no detectable loss of accuracy, provided the agent is told which services exist. They do not replace the agent's own tool calls, which need generation.

DATA AVAILABILITY

The harness (catalog generator, MCP server, LLM proxy, sandbox setup, runner), the 30 tasks with hidden tests and reference solutions, and the analysis scripts are available at https://anonymous.4open.science/r/XXXX. The experiments cost about $19 in API fees.

USE OF GENERATIVE AI

Claude Code, Anthropic's coding agent, helped throughout this work: it wrote the harness, catalog, tasks and analysis scripts, ran the experiments, and drafted the paper, under the authors' direction. The authors chose the research questions, reviewed the code, results and text, and are responsible for the content. The session analyzed in Sections III and VIII is this assistant's own session.

REFERENCES

[1] Model Context Protocol, "Model context protocol specification." https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[2] opencode contributors, "opencode: an open-source coding agent." https://opencode.ai, version 1.19.4, 2026.

[3] TypeSafe AI, "Jev 1.14 decision model." https://typesafe.ai; served via OpenRouter as `typesafe/jev-1.14`, 2026.

[4] Microsoft, "Not every decision needs an LLM: Getting started with Microsoft-Decision-1." https://techcommunity.microsoft.com/blog/educatordeveloperblog/not-every-decision-needs-an-llm-getting-started-with-microsoft-decision-1/4563468, Oct. 2026. Microsoft Community Hub, Educator Developer Blog.

[5] OpenRouter, "Decisions API reference." https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request, 2026.

[6] Artificial Analysis, "Artificial analysis decision index v1.0." https://artificialanalysis.ai/evaluations/artificial-analysis-decision-index, 2026.

[7] R. S. Olson, "When to use a decision model instead of an LLM." Blog post, 15 October 2026, https://www.randalolson.com/2026/10/14/when-to-use-a-decision-model-instead-of-an-llm/, 2026.

[8] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," *arXiv preprint arXiv:2307.16789*, 2023.

[9] H. Chen, X. Liu, J. Jiang, B. He, and X. Liu, "Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference," *arXiv preprint arXiv:2603.13426*, 2026.

[10] J. Lee, "Selection is retrieval, abstention is not: On-device tool routing over 70 Korean-English actions," *arXiv preprint arXiv:2609.18672*, 2026.

[11] H. Ross, A. S. Mahabaleshwarkar, and Y. Suhara, "When2Call: When (not) to call tools," in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[12] C. Qian, E. C. Acikgoz, H. Wang, X. Chen, A. Sil, D. Hakkani-Tür, G. Tur, and H. Ji, "SMART: Self-aware agent for tool overuse mitigation," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[13] Q. Wu, S. Das, M. Amani, A. Nag, S. Lee, K. P. Gummadi, A. Ravichander, and M. B. Zafar, "To call or not to call: A framework to assess and optimize LLM tool calling," *arXiv preprint arXiv:2605.00737*, 2026.

[14] C.-E. Sun, L. Liu, G. Yan, Z. Wang, and T.-W. Weng, "LLM agents already know when to call tools – even without reasoning," *arXiv preprint arXiv:2605.09252*, 2026.

[15] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 of *PMLR*, pp. 48371–48392, 2025.

[16] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[17] C. Bandi *et al.*, "MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers," *arXiv preprint arXiv:2602.00933*, 2026.

[18] J. Li *et al.*, "The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution," *arXiv preprint arXiv:2510.25726*, 2025.

[19] V. Bhat, J. Vaghasiya, M. A. Mohsin, and A. Aali, "Benchmarking the benchmarks: A validity audit of tool-calling evaluation," *arXiv preprint arXiv:2607.02577*, 2026.

[20] X. Wang *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *International Conference on Learning Representations*, 2025.

[21] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, and B. Yangel, "SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents," *arXiv preprint arXiv:2505.20411*, 2025.